DreamLog

Compression Enables Generalization in Logic Programming

Alexander Towell

10.5281/zenodo.19490027 (preprint)

Southern Illinois University Edwardsville

April 2026

The idea

Problem. Knowledge bases grow but do not learn. You can add 300 facts about a family tree, and the system still cannot tell you whether a new person is someone's father. The facts accumulate, understanding does not.

Claim. Compression *is* learning (Solomonoff's idea). The shortest KB consistent with the data generalizes best.

System. DreamLog, a Prolog variant with wake-sleep cycles inspired by DreamCoder.

- **Wake:** answer queries against the KB, and if a predicate is undefined, optionally ask an LLM to propose rules for it.
- **Sleep:** 8 compression operations, each verified with atomic rollback. Examples: prune facts already derivable from rules; compress repeated patterns as "rule + exceptions" (next slide); invent a new predicate when two existing ones share the same logical shape; LLM proposes cross-predicate rules like `father` $\leftarrow$ `parent` + `male`.

Every compression is behaviorally equivalent to the original KB on the verification suite. Nothing gets "worse."

What "compression discovers concepts" looks like

Pattern: compression via exceptions. Pick a default that covers most cases, list the exceptions. You gain compression whenever the exceptions are fewer than the non-exceptions.
From the crafting experiment, symbolic only (no LLM).
Input: seven ground facts, one per master artisan.

```
(master_artisan kael)    (master_artisan sera)
(master_artisan yara)    (master_artisan dax)
(master_artisan mira)    (master_artisan orin)
(master_artisan zara)
```

After one sleep cycle:

```
(master_artisan X) :- (artisan X),
                      (not (exception_master_artisan_artisan X)).
(exception_master_artisan_artisan thom).
(exception_master_artisan_artisan fen).
```

Seven facts become three clauses: "every artisan is a master unless listed." The system invents a fresh predicate named `exception_{head}_{guard}` to hold the two exceptions.
At test time, adding `(artisan venn)` makes `(master_artisan venn)` derivable. Nobody wrote that fact; the system classified a new entity using a concept it discovered from structure alone.

The result I care most about

The worked example above is one rule from the crafting experiment. Aggregate over 33 test checks (19 positive, 14 negative), 5 runs per condition:

Condition	Recall	Precision	Rules
No dream	0%	—	0
Raw LLM prompting	0%	—	0
Symbolic only	53%	59%	5
Full pipeline	$64 \pm 2\%$	64%	20

The raw LLM scores 0% because the terms are invented. Symbolic compression alone hits 53% without any LLM involvement, which means compression discovers real concepts from data structure, not from prior knowledge. The LLM adds another 11 points on top.

This is the cleanest evidence I have for the thesis.

The ask

The paper needs work before it's venue-ready: a head-to-head Popper or Metagol baseline, figures, and likely a larger-scale run.

What I'd want from you as coauthor.

- Venue guidance. What to target, what that committee cares about, what's realistic.
- A careful read on the paper. Does the argument land, is anything confusing, is the story coherent end to end?
- Methodology sanity check. Is the benchmark clearly framed, is the evaluation airtight, are there standard comparisons I should run that I've missed?
- Dissertation fit: chapter, or standalone paper toward my publication count? Both paths are real, I'd like your read on which makes more sense here.

What I'd do. All the technical work: implementation, statistics, the Popper benchmark run, figures, writing, revisions.